

PipelineInsights — Trello cards (copy/paste)

How to use: create a board with lists **Phase 1 Tennis**, **Phase 2 Tennis test**, **Phase 3 MLB**, **Phase 4 Generalize**, **Phase 5 NFL**, **Phase 6 Live**. For each card below, paste the **title** into the card name, paste the **description** into the card body, then add a checklist named **Done when** and paste the - [] lines.

Trello turns - [] into checkable items if you paste them into a checklist (or into the description as markdown).

Board rules (pin this on a “Rules” card if you want): models only read `player_game_features`; bump `modelversion` never overwrite; time-based splits only; dashboard only reads `vw_fact_player_prop_odds`.

LIST: Phase 1 Tennis (Weeks 1–8, Aug 19 – Oct 14, 2026)

CARD

Title: 1.1 Set up development environment

Description:

Goal: a working Python environment you can run code in.

Why: every later step (ETL, features, models) fails if the environment is messy. A `virtualenv` keeps this project’s packages from colliding with other class/work projects.

Checklist: Done when

- [] Python 3.10+ installed
- [] VS Code (or similar) installed
- [] Project folder created (e.g. `pipelineinsights/`)
- [] Virtual environment created (`python -m venv venv`)
- [] `venv` activated
- [] Installed: `pandas`, `numpy`, `scikit-learn`, `xgboost` or `lightgbm`, `requests`, `sqlalchemy`
- [] `python -c "import pandas; print(pandas.__version__)"` prints a version with no errors

CARD

Title: 1.2 Set up the database

Description:

Goal: a real database (not only CSVs) so data can grow and be queried.

Why: models and the dashboard will join players, matches/games, odds, features, and predictions. A DB is the single source of truth.

Start with PostgreSQL if you can; SQLite is OK for a lighter first pass.

Checklist: Done when

- ☐ PostgreSQL installed (or SQLite chosen on purpose)
- ☐ Database created (e.g. pipelineinsights_db)
- ☐ Python connects with SQLAlchemy
- ☐ `SELECT 1;` from Python returns a result with no connection errors

CARD

Title: 1.3 Design core tables

Description:

Goal: schema on paper, then `CREATE TABLE`, before loading data.

Tables to sketch: players, matches, odds, player_prop_predictions. Keep it simple; add columns later.

Also plan (even if you build it in 1.7): player_game_features as the only table models will read.

Checklist: Done when

- ☐ Column names and types sketched for players
- ☐ Column names and types sketched for matches
- ☐ Column names and types sketched for odds
- ☐ Column names and types sketched for player_prop_predictions (include modelversion, predictiontimestamp)
- ☐ `CREATE TABLE` SQL run for each table
- ☐ `SELECT * FROM players;` (and the other tables) returns empty results with no errors

CARD

Title: 1.4 Pull first tennis data

Description:

Goal: real ATP/WTB history in `players` and `matches`.

Source: Jeff Sackmann ATP (or WTA) match CSVs on GitHub.

Checklist: Done when

- ☐ Sackmann CSV downloaded
- ☐ Script reads one CSV with pandas
- ☐ `df.head()` printed and columns understood
- ☐ Columns mapped into `players` + `matches`
- ☐ Insert script loads data
- ☐ Database query shows real player names and match results

CARD

Title: 1.5 Pull odds data (basic)

Description:

Goal: some historical or current odds linked to matches.

Example source: The Odds API free tier. First print raw JSON so you see the shape, then parse.

Checklist: Done when

- ☐ Odds API account / key created
- ☐ Script calls API and prints raw JSON
- ☐ JSON parsed into odds table rows
- ☐ Rows inserted
- ☐ Odds rows link to real match IDs

CARD

Title: 1.6 First feature: rolling average

Description:

Goal: last-N match average for one stat (e.g. games won). This is the sanity-check idea behind the baseline model.

Why: if this number is wrong, every model downstream is wrong.

Checklist: Done when

- ☐ Match data grouped by player, sorted by date
- ☐ 5-match rolling mean computed
- ☐ One real player eyeball-checked against last 5 matches
- ☐ Number makes sense by hand

CARD

Title: 1.7 Full player_game_features table

Description:

Goal: the single feature table all models will read. Sport-specific work stops at ETL + this table.

Add: rolling 3/5/10, head-to-head, rest days, surface.

Checklist: Done when

- ☐ Rolling windows 3, 5, 10
- ☐ Head-to-head history column(s)
- ☐ Rest days since last match
- ☐ Surface (clay/grass/hard)
- ☐ Written to player_game_features keyed by player + match
- ☐ Query for a player-match shows sensible numbers
- ☐ No nulls on matches that have enough history

CARD

Title: 1.8 Baseline model

Description:

Goal: interpretable rolling-average prediction, adjusted for context (e.g. surface). Every other model must beat this.

Store as `modelversion = baseline_v1_points` (or the stat you pick). Never overwrite versions.

Checklist: Done when

- ☐ Function: `baseline = avg_last_5 * context_adjustment`
- ☐ Simple surface (or similar) adjustment only — not overcomplicated
- ☐ Run across feature table
- ☐ Inserted into player_prop_predictions with `modelversion + predictiontimestamp`
- ☐ Predictions look plausible vs player history

CARD

Title: 1.9 GBM model (point estimate)

Description:

Goal: first real ML model (XGBoost or LightGBM).

Split by date: train on earlier matches, test on later matches. Never shuffle randomly (that is lookahead bias).

Success = MAE lower than baseline on the same test set.

Checklist: Done when

- ☐ Time-based train/test split (not random)
- ☐ Target column chosen (e.g. games/points)
- ☐ Model fit on train
- ☐ Predict on test
- ☐ MAE calculated
- ☐ MAE is better than baseline on the same split

CARD

Title: 1.10 GBM quantile CIs (lowerci / upperci)

Description:

Goal: a range, not only a point. Train extra quantile models (e.g. 20th and 80th percentile) for lowerci and upperci.

Store mean + lower + upper as `gbm_v1_points`.

Checklist: Done when

- ☐ Quantile model for ~20th percentile (lowerci)
- ☐ Quantile model for ~80th percentile (upperci)
- ☐ Mean + lowerci + upperci stored with modelversion `gbm_v1_points`
- ☐ Actual outcomes fall in the band about ~60% of the time (20/80 window)

CARD

Title: 1.11 Ensemble (weighted average)

Description:

Goal: combine baseline + GBM. Weight inversely to error (better model gets more weight).

Store as `ensemble_v1_points`. Stacked meta-model is a later stretch goal, not this card.

Checklist: Done when

- ☐ Historical MAE for baseline and GBM
- ☐ Weights set (inversely to error)
- ☐ $\text{ensemble} = w_{\text{baseline}} * \text{baseline} + w_{\text{gbm}} * \text{gbm}$
- ☐ Stored with modelversion ensemble_v1_points
- ☐ Ensemble MAE on test is better than either model alone

CARD

Title: 1.12 Reusable backtesting harness

Description:

Goal: one evaluation loop you reuse for every sport and every modelversion: time-based split, MAE, RMSE, later edge-vs-market.

Must be a module, not a one-off notebook.

Checklist: Done when

- ☐ Function takes a date range
- ☐ Pulls predictions + actuals
- ☐ Computes MAE and RMSE
- ☐ Works for any modelversion with no hardcoded sport logic
- ☐ Results saved to CSV or a table
- ☐ Same script run on baseline_v1, gbm_v1, ensemble_v1 with three comparable scores

CARD

Title: Milestone — Phase 1 tennis foundation (Oct 14, 2026)

Description:

Reference implementation of the generic pipeline on tennis (fewest moving parts: no roster, no injury ripple, no possession sharing).

Checklist: Done when

- ☐ Real tennis data in DB
- ☐ player_game_features populated
- ☐ baseline_v1, gbm_v1, ensemble_v1 stored
- ☐ Backtester proves ensemble beats baseline
- ☐ modelversion + predictiontimestamp conventions in use

LIST: Phase 2 Tennis test (Weeks 9–14, Oct 15 – Nov 25, 2026)

CARD

Title: 2.1 Multi-season tennis backtests

Description:

Weeks 9–10. Stress-test, don't build. Compare baseline vs GBM vs ensemble vs market line across 2–3 seasons.

Checklist: Done when

- ☐ Backtester run on at least 2–3 full seasons
- ☐ Table/chart of MAE per model per season
- ☐ Performance looks consistent, not one lucky season
- ☐ Market line included if odds exist

CARD

Title: 2.2 Tune ensemble weights

Description:

Try weight mixes (e.g. 30/70, 50/50, 70/30). Choose using out-of-sample MAE. Bump modelversion when weights change.

Checklist: Done when

- ☐ Several weight combos tested
- ☐ Best combo identified by out-of-sample MAE
- ☐ Written note of why that mix won
- ☐ New ensemble modelversion saved (old one kept)

CARD

Title: 2.3 Optional Bayesian / hierarchical layer

Description:

Weeks 11–12. Player random effects for low-data players. Build only to decide keep vs drop — do not build by default going forward if it doesn't help.

Checklist: Done when

- ☐ Lowest-data players identified
- ☐ Simple hierarchical model tried (e.g. PyMC or bambi)

- ☐ Accuracy compared vs GBM on those players only
- ☐ Written yes/no: meaningful improvement?
- ☐ If no: documented and not expanded

CARD

Title: 2.4 Stress-test tennis edge cases

Description:

Weeks 13–14. Retirements, surface transitions (clay→grass), debut/sparse history. This list becomes the next-sport checklist.

Checklist: Done when

- ☐ Retirement/mid-match stop matches pulled
- ☐ Surface-transition matches pulled
- ☐ Debut / short-history players pulled
- ☐ Model errors reviewed on those rows
- ☐ Written failure-mode checklist for MLB/NFL

CARD

Title: Milestone — Phase 2 tennis validated (Nov 25, 2026)

Description:

A validated, backtested tennis model with documented accuracy and a clean modular codebase.

Checklist: Done when

- ☐ Accuracy metrics documented
- ☐ Known weaknesses list written
- ☐ Code is modular enough to point at a new feature table later

LIST: Phase 3 MLB (Weeks 15–24, Nov 26, 2026 – Jan 27, 2027)

CARD

Title: 3.1 Add generic teams and games tables

Description:

Weeks 15–17. games replaces 1v1 matches (two teams). Design generically: home/away, date, rest,

travel — same shape must serve NFL later without a rebuild.

Checklist: Done when

- ☐ teams table created
- ☐ games table created (home/away, date, rest, travel fields)
- ☐ rosters/schedule concept exists
- ☐ Design notes say how this maps to NFL
- ☐ You can query a schedule-shaped table without errors

CARD

Title: 3.2 Pull MLB data

Description:

Use pybaseball (Statcast / game logs). Lahman is the free historical complement. Start with one season.

Checklist: Done when

- ☐ pybaseball (or equivalent) working
- ☐ One MLB season pulled first
- ☐ Player game logs in the database
- ☐ Query shows real MLB names and dates

CARD

Title: 3.3 Team-context features on player_game_features

Description:

Weeks 18–20. Do not let models read raw MLB tables. Extend the same feature table: opponent strength, park/venue, pace-equivalent metrics, pitcher-batter matchup.

Reuse baseline/GBM/ensemble almost unchanged — only feature inputs change.

Checklist: Done when

- ☐ Opponent strength feature
- ☐ Park/venue factor
- ☐ Pace-like / team-context metric
- ☐ Pitcher-batter (or equivalent) matchup stats
- ☐ Feature rows have player-level AND team-context columns
- ☐ Models still only read player_game_features

CARD

Title: 3.4 Injury / teammate availability features

Description:

Weeks 21–22. Tennis didn't need this; NFL will. Example: starting pitcher late scratch.

Checklist: Done when

- ☐ Free injury source identified
- ☐ injuries table built
- ☐ Feature like “key teammate out today”
- ☐ One known real absence matches the flag

CARD

Title: 3.5 Reuse model code on MLB

Description:

Point existing baseline.py, gbm_model.py, ensemble.py at MLB features. Change the feature list only, not model logic.

Checklist: Done when

- ☐ Same baseline script runs on MLB features
- ☐ Same GBM script runs on MLB features
- ☐ Same ensemble script runs on MLB features
- ☐ MLB rows in player_prop_predictions with new model versions
- ☐ No sport-specific if/else stuffed into model files

CARD

Title: 3.6 Backtest MLB (same rigor as tennis)

Description:

Weeks 23–24. Reuse the Step 1.12 harness.

Checklist: Done when

- ☐ Time-based MLB train/test
- ☐ MAE/RMSE for baseline, GBM, ensemble
- ☐ Format comparable to tennis report
- ☐ Ensemble vs baseline documented

CARD

Title: Milestone — Phase 3 MLB + translation playbook (Jan 27, 2027)

Description:

Working team-sport model AND a written “what changed” diff tennis → MLB. That diff is the playbook for NFL.

Checklist: Done when

- ☐ MLB model working
- ☐ Diff doc: schema changes
- ☐ Diff doc: feature changes
- ☐ Diff doc: what model code did NOT change
- ☐ Injury layer called out as the new piece tennis lacked

LIST: Phase 4 Generalize (Weeks 25–30, Jan 28 – Mar 10, 2027)

Do not skip this list to “get to NFL sooner.”

CARD

Title: 4.1 Feature lists in config files

Description:

Per-sport JSON/YAML feature sets. No hardcoded feature columns in model scripts.

Checklist: Done when

- ☐ tennis feature config file
- ☐ mlb feature config file
- ☐ Scripts read features from config
- ☐ Switching sport = change config, not code

CARD

Title: 4.2 Abstract model layer

Description:

baseline / GBM / ensemble / optional Bayesian accept any sport’s feature table. Only hyperparameters and feature lists vary.

Checklist: Done when

- ☐ baseline.py has no sport-specific logic

- ☐ gbm_model.py has no sport-specific logic
- ☐ ensemble.py has no sport-specific logic
- ☐ Same script runs tennis vs MLB by arguments/config only

CARD

Title: 4.3 Generic reporting view template

Description:

vw_fact_player_prop_odds parameterized per sport. Dashboard will only ever read this view.

Checklist: Done when

- ☐ View joins predictions + odds + player/game context
- ☐ Works for tennis rows
- ☐ Works for MLB rows
- ☐ Template/notes for a third sport

CARD

Title: 4.4 Write “add a new sport” onboarding doc

Description:

Internal doc: 1) data source 2) ETL 3) feature config 4) run existing model scripts 5) validate with backtester.

This is what makes NFL (and later sports) fast.

Checklist: Done when

- ☐ Doc written
- ☐ Five-step path is explicit
- ☐ Future-you could follow it without asking questions
- ☐ Architecture rule restated: sport-specific work only touches sources → ETL → raw tables → features

CARD

Title: Milestone — Phase 4 sport-agnostic framework (Mar 10, 2027)

Checklist: Done when

- ☐ Config-driven features
- ☐ Shared model code
- ☐ Shared reporting view pattern

- ☐ Onboarding doc complete
- ☐ Proven on tennis (individual) + MLB (team)

LIST: Phase 5 NFL (Weeks 31–42, Mar 11 – Jun 2, 2027)

CARD

Title: 5.1 NFL data acquisition + schema mapping

Description:

Weeks 31–33. Free sources: nflverse / nflfastR via nfl_data_py; ESPN / API-Sports free tiers if needed. Map into the generalized framework — do not invent a new NFL-only schema.

Checklist: Done when

- ☐ nfl_data_py (or nflverse) pull working
- ☐ Play-by-play and/or player game logs loaded
- ☐ Mapped into existing players / games / teams / odds pattern
- ☐ Query shows real NFL player stats

CARD

Title: 5.2 NFL feature config

Description:

Weeks 34–36. New work belongs here, not in model code: 17-game season / small sample, weather, injury reports, positional matchups (WR vs CB).

Checklist: Done when

- ☐ NFL feature config written
- ☐ Weather feature
- ☐ Injury-report severity feature
- ☐ Positional matchup feature
- ☐ Feature table fills from config without editing model scripts

CARD

Title: 5.3 Train and tune NFL baseline / GBM / ensemble

Description:

Weeks 37–39. Extra tuning time vs tennis/MLB because short seasons overfit easily. Quantile CIs still required. Time-based splits only.

Checklist: Done when

- ☐ baseline_v1 NFL version stored
- ☐ gbm_v1 NFL with lowerci/upperci stored
- ☐ ensemble_v1 NFL stored
- ☐ Time-based test, not random
- ☐ Ensemble beats baseline (even if the gap is smaller than tennis/MLB)

CARD

Title: 5.4 Full NFL historical backtest + market / ROI

Description:

Weeks 40–42. Multiple seasons, market-line comparison, hypothetical edge/ROI. Same report shape as tennis and MLB.

Checklist: Done when

- ☐ Multi-season backtest
- ☐ MAE/RMSE by modelversion
- ☐ Compared to sportsbook lines
- ☐ Edge/ROI simulation noted (with the usual “hypothetical / not betting advice” caution)
- ☐ Performance report matches tennis/MLB format

CARD

Title: Milestone — Phase 5 NFL model (Jun 2, 2027)

Checklist: Done when

- ☐ Working NFL model on reused architecture
- ☐ Versions tagged, old versions kept
- ☐ Overfitting risks from short seasons documented

LIST: Phase 6 Live (Weeks 43–54, Jun 3 – Sept 1, 2027)

CARD

Title: 6.1 Dashboard sport-switcher

Description:

Weeks 43–46. Extend Dash/UI for tennis, MLB, NFL. UI reads only `vw_fact_player_prop_odds`.

Checklist: Done when

- ☐ View extended for NFL
- ☐ Sport switcher in UI
- ☐ Tennis toggle shows tennis rows
- ☐ MLB toggle shows MLB rows
- ☐ NFL toggle shows NFL rows
- ☐ Dashboard does not query raw tables

CARD

Title: 6.2 Live pipeline test (scheduler)

Description:

Weeks 47–50. Real preseason/early slate: daily ETL, feature refresh, scoring. Scheduler (e.g. APScheduler). Not just historical backtests.

Checklist: Done when

- ☐ Scheduler runs ETL
- ☐ Features refresh automatically
- ☐ Models score automatically
- ☐ Predictions land in player_prop_predictions with timestamps
- ☐ Fresh rows appear in the dashboard without manual script runs
- ☐ At least one real week of NFL (or preseason) slate exercised

CARD

Title: 6.3 Buffer — fix what live breaks

Description:

Weeks 51–54. Deliberate slack. Earlier phases had none. Live sports feeds break in ways backtests miss: API rate limits, schema drift, odds formatting.

Checklist: Done when

- ☐ Running bug log started
- ☐ API/rate-limit issues handled or documented
- ☐ Schema-drift / missing-field issues fixed
- ☐ Odds-feed formatting issues fixed
- ☐ Recalibration done if live error looks systematically off
- ☐ Pipeline runs a full week with no manual intervention

CARD

Title: Final milestone — NFL live (Sept 1, 2027)

Description:

Live NFL player-prop pipeline on architecture proven first on tennis, then MLB, then a config-driven generalize step.

Checklist: Done when

- ☐ NFL predictions generating on a schedule
- ☐ Dashboard live with sport switcher
- ☐ Unattended week achieved
- ☐ Architecture still portable (new sport = source + ETL + feature config)
- ☐ All modelversions retained for side-by-side comparison

Optional labels to add on every card

- Sport: Tennis / MLB / NFL / Shared
- Type: ETL / Features / Model / Backtest / Docs / Dashboard
- Rule: no-raw-reads / time-split / version-bump